

Д31 - Подбор вариантов тестирования в кейсах

Выбранный сервис

Для задания выбрал **Booking-сервис** (бронирование спортивных мероприятий).

Ниже по каждому из 10 кейсов указал: - какие виды тестирования нужны и зачем; - на каком уровне лучше проверять (unit, API, UI); - какие позитивные и негативные сценарии считаю обязательными.

1) Регистрация нового пользователя

Регистрация - это “входная дверь” в продукт, поэтому здесь нельзя ограничиваться только UI-проверками.

Виды тестирования и почему: - **Функциональное** - проверяем обязательные поля, формат email, подтверждение пароля. - **Интеграционное** - для авторизации через Google/Facebook важно убедиться, что OAuth-поток корректно завершает создание аккаунта. - **Security** - контроль хэширования пароля, защита от брутфорса и корректная валидация внешних токенов.

Уровни тестов: - unit: правила валидации email/пароля и совпадение паролей. - API: эндпоинты регистрации и OAuth-callback, ошибки и коды ответа. - UI: поведение формы, сообщения пользователю, переходы после успеха/ошибки.

Позитивные сценарии: - Успешная регистрация по email и паролю. - Успешная регистрация/логин через Google/Facebook.

Негативные сценарии: - Невалидный email, слабый пароль, пароли не совпадают. - Повторная регистрация с уже существующим email. - Пользователь отменил OAuth или пришел невалидный токен от провайдера.

2) Поиск мероприятий

Поиск влияет на конверсию в бронирование: если фильтры работают плохо, пользователь просто не найдет нужное событие.

Виды тестирования и почему: - **Функциональное** - корректность фильтров по виду спорта, дате и местоположению. - **Интеграционное** - соответствие ответов данным из БД/поискового слоя. - **Performance** - приемлемое время ответа при большом количестве мероприятий.

Уровни тестов: - unit: логика применения нескольких фильтров, сортировка, расчет пагинации. - API: ответы списка событий с фильтрами и пагинацией. - UI: взаимодействие с фильтрами и переходы по страницам.

Позитивные сценарии: - Поиск только по одному фильтру. - Поиск по комбинации фильтров (например, спорт + дата + локация). - Корректная пагинация (первая/средняя/последняя страница).

Негативные сценарии: - Неверный формат даты. - Запрошена страница вне диапазона. - Пустая выдача отображается корректно (без ошибок интерфейса).

3) Бронирование билетов

Здесь основной риск - некорректные остатки и гонки при параллельных покупках.

Виды тестирования и почему: - **Функциональное** - правильное создание брони и списание доступных мест. - **Интеграционное** - консистентность между сервисом бронирования и хранилищем остатков. - **Нагрузочное** - устойчивость при всплесках одновременных броней.

Уровни тестов: - unit: проверка допустимого количества билетов и бизнес-ограничений. - API: сценарии с разными остатками и конкурентными запросами. - UI: выбор количества мест, блокировка невалидного количества, тексты ошибок.

Позитивные сценарии: - Успешная бронь доступного числа билетов. - Параллельные брони не приводят к “перепродаже” мест.

Негативные сценарии: - Запрос количества больше остатка. - Остаток = 0. - Места закончились между открытием формы и подтверждением брони.

4) Оплата билетов

Это самый критичный денежный флоу: здесь важны статусы, надежность и безопасность.

Виды тестирования и почему: - **Функциональное** - правильная смена статусов оплаты/брони. - **Интеграционное** - стабильная работа с PayPal/Stripe и вебхуками. - **Security** - проверка подписи вебхуков и идиempотентности (чтобы не было двойного списания).

Уровни тестов: - unit: преобразование внешних статусов во внутренние. - API: платежные эндпоинты и обработка вебхуков. - UI: пользовательский путь “оплатить -> вернуться -> увидеть итоговый статус”.

Позитивные сценарии: - Успешная оплата и перевод брони в PAID. - Повторная доставка одного и того же вебхука не меняет состояние повторно.

Негативные сценарии: - Платеж отклонен банком/провайдером. - Пользователь прервал оплату. - Пришел вебхук с невалидной подписью.

5) Промокоды и скидки

В этом кейсе частые ошибки - неверная математика скидки и пропуск граничных условий.

Виды тестирования и почему: - **Функциональное** - верный расчет скидки (процент/фикс) и итоговой суммы. - **Интеграционное** - получение актуального состояния промокода (активен, истек, лимит исчерпан). - **Граничное** - срок действия, минимальный чек, лимит применений.

Уровни тестов: - unit: формулы, округления, правило “итог не может быть меньше нуля”. - API: применение промокода к заказу и валидационные ошибки. - UI: ввод промокода, пересчет суммы, пользовательские сообщения.

Позитивные сценарии: - Корректное применение процентной скидки. - Корректное применение фиксированной скидки.

Негативные сценарии: - Просроченный/неактивный промокод. - Промокод не подходит под условия акции. - Скидка больше стоимости заказа (результат должен быть 0, не отрицательное число).

6) Оставление отзыва

Отзывы одновременно влияют на доверие пользователей и являются точкой риска по безопасности.

Виды тестирования и почему: - **Функциональное** - оставить отзыв может только пользователь, реально посетивший событие. - **Security** - защита от XSS и других инъекций. - **Валидационное** - границы длины текста и допустимый диапазон рейтинга.

Уровни тестов: - unit: санитизация/экранирование текста, правила рейтинга. - API: права на создание отзыва и ошибки валидации. - UI: форма отзыва, ограничение длины, подсказки по ошибкам.

Позитивные сценарии: - Пользователь после посещения оставляет отзыв и рейтинг 1–5. - Отзыв корректно показывается в карточке мероприятия.

Негативные сценарии: - Попытка отправить скрипт в тексте комментария. - Слишком длинный комментарий. - Пользователь, который не посещал событие, пытается оставить отзыв.

7) Уведомления о мероприятиях

Ошибки в уведомлениях бьют по пользовательскому опыту: можно пропустить событие или получить дубли.

Виды тестирования и почему: - **Функциональное** - отправка email/SMS в правильное время. - **Интеграционное** - взаимодействие с SMTP/SMS-провайдером и обработка их ошибок. - **Надежность** - retry и защита от повторной отправки одного уведомления.

Уровни тестов: - unit: шаблоны уведомлений, расчеты времени с учетом таймзоны. - API: очереди/сервис уведомлений, статусы доставки. - UI: настройки каналов и частоты уведомлений.

Позитивные сценарии: - Напоминание приходит вовремя, данные в шаблоне корректны. - После временного сбоя провайдера уведомление доходит по retry.

Негативные сценарии: - Невалидные контактные данные (email/телефон). - Повторный запуск джобы не создает дубль уведомления. - Ошибка таймзоны не сдвигает напоминание на неверное время.

8) Личный кабинет организатора

Главный риск - нарушение прав доступа, когда организатор может менять не свои события.

Виды тестирования и почему: - **Функциональное** - создание и редактирование события. - **Security / Access Control** - проверка ролей и ограничений на чужие сущности. - **Интеграционное** - корректное сохранение и отображение изменений в каталоге.

Уровни тестов: - unit: валидация даты, цены и обязательных полей. - API: CRUD операций организатора + проверки RBAC. - UI: формы создания/редактирования и блокировка запрещенных действий.

Позитивные сценарии: - Организатор создает свое событие. - Организатор успешно редактирует свои данные события.

Негативные сценарии: - Попытка редактировать чужое событие. - Ввод некорректных данных (дата в прошлом, отрицательная цена). - Неавторизованный доступ в кабинет организатора.

9) Система отзывов о пользователях

Кейс чувствительный с точки зрения репутации, поэтому нужна и техническая, и процессная защита.

Виды тестирования и почему: - **Функциональное** - корректное добавление метки с причиной. - **Security/Compliance** - предотвращение злоупотреблений, проверка прав на удаление/оспаривание. - **Аудит** - хранение истории изменений: кто, когда и по какому основанию оставил метку.

Уровни тестов: - unit: валидация причины, длины комментария, допустимых статусов. - API: создание/просмотр/изменение/удаление по ролям. - UI: ввод метки, отображение истории, кнопка апелляции (если есть в требованиях).

Позитивные сценарии: - Организатор оставляет обоснованную метку участнику своего мероприятия. - Пользователь видит статус метки и результат апелляции.

Негативные сценарии: - Пустая или невалидная причина. - Организатор пытается оставить метку пользователю, который не связан с его событием. - Попытка удалить чужую метку без нужной роли.

10) Отображение календаря мероприятий

Календарь - это визуализация данных, где часто возникают ошибки на границах дат и таймзон.

Виды тестирования и почему: - **Функциональное** - корректная работа дневного, недельного и месячного видов. - **Интеграционное** - правильная загрузка событий в заданный диапазон дат. - **UI/UX** - предсказуемая навигация и корректное отображение в разных локалях.

Уровни тестов: - unit: вычисление диапазонов дат и переходы между периодами. - API: выдача событий по временному интервалу. - UI: переключение режимов, навигация вперед/назад, кнопка "сегодня".

Позитивные сценарии: - События корректно отображаются в day/week/month. - Переход по датам не теряет события и не дублирует их.

Негативные сценарии: - Границы: конец месяца, переход года, високосная дата. - Таймзона/локаль смещает событие на неверный день. - Пустой диапазон обрабатывается корректно (чистый календарь, без ошибок).

Приоритеты тестирования

Если ограничено время, в первую очередь проверял бы: - регистрацию/авторизацию; - бронирование; - оплату; - роли и права доступа.

Во вторую очередь: - поиск и фильтры; - промокоды; - уведомления; - календарь.

И отдельно не забывал бы про “длинные хвосты”: - отзывы и пользовательские метки; - граничные сценарии и валидации в UI.